



F1R3FLY.IO

F1R3GAMES · GAME DESIGN

F1R3Beat

Collective rhythmic intelligence on a F1R3FLY shard

Version 2 · 6 October 2026

Conforms to F1R3Node-Rust master@95be0d4, the F1R3Games Portal at F1R3Games
main@5937e4e,
and the score calculus as implemented in F1R3Score@76a355d

SUMMARY

F1R3Beat is a multiplayer game in which a community makes a beat together. The board is a grid: time runs left to right in columns, each a fixed subdivision of the beat, over a time signature and a number of bars, all chosen when the game is set up, and each row is one instrument of a rock band. Each player controls exactly one cell, and the only thing a player can do on the board is choose what that cell plays: nothing, or one pitch from the row's palette. Players can also message one another and send one another F1R3Cap. Any groove larger than one cell therefore has to be negotiated.

Patterns made in play are published to a gallery, and the ones people engage with most are bred. A pattern is written as a score in the calculus of *Scores as Processes*, as implemented in F1R3Score: the beat is the parallel composition of its instruments' lines. Reproduction is defined on those terms. A child takes one parent's rhythm and the other's melody, row by row, or takes whole lines from each parent, so recombination happens exactly where the score composes.

This document specifies the game against the stack as it stands on 6 October 2026. It replaces the game installed with the Portal, an 8×16 sequencer in which any participant could toggle any cell. It reuses the messaging and payments that the F1R3Pix design added to the Portal. Version 2 records the review of 6 October 2026. All sixteen decisions in Section 15 are agreed. One changed in review: every note lasts one column, and the column is chosen when the game is set up (D5).

CONTENTS

1 Purpose, sources and conformance	2
2 What the review found	3
3 The game	4
4 Architecture	7
5 The F1R3Beat environment	8
6 Messages and payments	10
7 From grid to score	11
8 The client	13
9 Galleries	15
10 Evolution	16
11 Security and abuse	19
12 Manifest and conformance	20
13 F1R3Gaze	20
14 Development plan	21
15 Decisions	22

1 PURPOSE, SOURCES AND CONFORMANCE

1.1 What the game is for

F1R3Beat belongs to the F1R3Games suite of collective intelligence games, *E Pluribus Sapiens*. The suite's premise is that the actions of any one player are severely limited, except for communication: anyone can talk to anyone, and everyone can see, or here hear, the global result of the collective effort. Tokens generalise capital. They amplify or attenuate what a player can achieve, and their first use is in negotiation.

F1R3Beat is cut from the same cloth as F1R3Pix. Where F1R3Pix makes coordination visible as an image, F1R3Beat makes it audible as a groove. A kick on every beat, a backbeat on two and four, or a bass figure that locks with the kick each needs several players to agree. The original prototype already sketched the incentive: “I’ll pay 100 F1R3Cap if we get four-on-the-floor going.”

F1R3Beat differs from F1R3Pix in one respect that shapes much of this document. Its plays evolve. As with F1R3Skein’s tunes, patterns that attract more engagement are selected more often for reproduction, so the gallery is a population, and the community’s taste acts on it as selection pressure.

1.2 Sources reviewed

- The original F1R3Beat design session (9 March 2026). It specified the rectangular grid, the time signature and bars, sixteenth-note steps, the rock-band timbres (bass, a standard drum set, guitar, keyboard, and a saxophone in place of vocals), one cell per player, and messaging and tokens as in F1R3Pix.
- The music-snippet evolution session (3 March 2026) and F1R3Skein/breeding, which define F1R3Skein’s breeding: engagement-weighted selection, the Mother/Father crossover of pitch and duration streams, padding by rests as mutation, and culling of the least engaged.
- The design session of 6 October 2026, in which the author of the game settled the pitch palette, the single drum row, local tempo, and the use of F1R3Score’s notation for reproduction (D4, D7 and D9 below), and the review of version 1 later that day, which agreed every decision, set note length by column (D5), and accepted the operators of D12 as workable for now.
- The F1R3Games repository at `main@5937e4e`. It holds the March prototype (F1R3Beat/UI, F1R3Beat/Shard), the Portal (`Portal/f1r3games-portal`), and F1R3Pix as merged on 6 October: the client, the seat/paint/say environment, the Portal’s payments domain, and host protocol 2.
- The F1R3Pix design document, version 1 (5 October 2026), whose structure and many of whose decisions this document follows.
- F1R3Score at `76a355d` (28 September 2026), the player for *Scores as Processes*: its guide, `DESIGN.md`, `std.score` and examples. The player was built and run for this document. Every score in it was checked with `f1r3score check`, and the worked example was played with `f1r3score play`.
- F1R3Node-Rust master@`95be0d4` and Embers @`28e50d4`, through the Portal and F1R3Pix designs.
- The brand portal, `github.com/F1R3FLY-io/f1r3fly-brand-portal` at `283e00e` (brand kit v1.2), which governs this document’s design.

1.3 Order of authority

Where sources disagree, this document follows the order the Portal design set: F1R3Node-Rust master; then Embers; then F1R3Sky; then F1R3Gaze and ign1t10n; then the Portal and F1R3Pix as merged on main. For the meaning of a score, F1R3Score is authoritative. The March prototype and the earlier F1R3Games contracts are research. They are the record of what the game is meant to be, and this document keeps their rules. It does not keep their storage, signing or contract code.

1.4 Conventions

MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. Requirements are numbered R1, R2, ... and decisions D1, D2, A *participant* is an address in an instance's participants map in the Portal environment. A *seated* participant is one who has been given a cell. A *pattern* is the content of the board at one moment, written as a score. A *column* is the subdivision of the beat chosen when the game is set up, and a *step* is one column. Durations are in whole notes, as in F1R3Score.

2 WHAT THE REVIEW FOUND

FINDING 1 The installed game environment models a different game

The Portal's templates/games/f1r3beat.rho defines toggle(instance, voice, step, on) on a fixed pattern of 8 voices by 16 steps. Any participant may toggle any cell. That is a shared drum machine. F1R3Beat has one cell per player, set only by its owner. The drift is the same one F1R3Pix found: the branding branch rewrote the March prototype to a single global grid, and the Portal followed that branch.

FINDING 2 The pattern has no time signature or length

The March design has the community pick a time signature and a number of bars, with each bar measured in sixteenth notes. Every implementation since fixes the pattern at 16 steps.

FINDING 3 Cells cannot carry a pitch, so the drum set took eight rows

The prototype's cells are on or off. With no way to say which note a cell plays, the drum set was spread over eight rows (kick, snare, hats, toms, clap, rim), and the bass, guitar, keyboard and saxophone were never given rows at all. A pitched instrument cannot be played with a cell that only says "on".

FINDING 4 Tempo is a shared, global move

The installed environment has tempo(instance, bpm), which any participant can set for everyone. The author of the game has settled that tempo is local to each listener (D7).

FINDING 5 Messaging and payments now exist, and F1R3Beat can reuse them

F1R3Pix, merged on 6 October, added sealed messages (say, mail, outbox), the Portal environment's payments.send and payments.list, and host protocol 2 with pay, open and declared capabilities. None of these is specific to F1R3Pix. F1R3Beat adopts them unchanged, with its own domain strings.

FINDING 6 Breeding exists only in F1R3Skein's prototype

F1R3Skein/breeding/snippet-evolution.jsx breeds snippets of two streams, pitches and durations. Its children pair one parent's pitches with the other's durations, and two more convert one parent's stream into the other kind by rewriting digits in base 5 or base 22. For F1R3Beat, the Portal records only a parents field in the gallery header. No operator is defined on a beat.

FINDING 7 A shared keyboard does not keep a grid's time

F1R3Score models a player at an instrument as a *touch* (a message holding a duration) sent to a *key* (a receipt holding a pitch). That suggests one keyboard per row, with each cell a touch entered at its step. Running that encoding showed why it fails. A key is one receipt, so every touch waiting at it is in its contention set whatever its stamp, and the contention set is resolved by weight, not by time. With touches at steps 0, 8 and 10 of one kick, the player sounded the kick at steps 8, 10 and 11, not at step 0. A grid's cells therefore become notes of a *line*, one line per row, as §7 specifies. That encoding plays every cell at its step.

3 THE GAME

3.1 Rules

1. The arena is a grid with one row per instrument and one column per step of the subdivision. Its time signature, number of bars and column size are chosen when the instance is created and do not change.
2. Each participant may be *seated* at exactly one cell, and each cell seats at most one participant. A cell with no one seated is *void*. It is silent, unless the instance was seeded from a pattern (D16).
3. The only move on the board is *set*: a seated participant chooses what their own cell plays: nothing, or one pitch from the row's palette. Every note lasts one column.
4. Off the board, any participant may send a message to any set of other participants, and may send F1R3Cap to any set of other participants.
5. Each player hears the pattern loop at a tempo of their own choosing. The tempo at which a player is listening is part of how other players see them.
6. Everything that happens on the board is recorded. The pattern at any moment can be published, and published patterns breed.

There is no win condition inside a single game. Goals come from outside: from what the players agree to make together, from the gallery's selection, or later from a tournament.

3.2 The grid

The rows are fixed in this version, in this order from top to bottom:

row	index	timbre	F1R3Score declaration	range
drums	0	General MIDI percussion	drums = gm(0) on 10	the 12 pieces of the kit (Table 1)
bass	1	Electric bass (finger)	bass = gm(33) on 2	E1–G3 (MIDI 28–55)
guitar	2	Overdriven guitar	guitar = gm(29) on 3	E2–E5 (40–76)
keys	3	Acoustic grand piano	keys = gm(0) on 4	C2–C6 (36–84)
sax	4	Tenor saxophone	sax = gm(66) on 5	G#2–D#5 (44–75)

A column has length u whole notes, chosen from $1/4$, $1/8$, $1/16$ and $1/32$ and the triplet subdivisions $1/6$, $1/12$ and $1/24$. A time signature n/d with column u gives $n/(du)$ steps per bar, which must be a whole number. With b bars the pattern has $S = nb/(du)$ steps and $5S$ cells. A cell is addressed by its step t and row ρ , and its index is $c = 5t + \rho$, so cells are numbered in time order and simultaneous cells are adjacent. That order is the one encodings use, and the one keyboard navigation follows.

time signature	4/4	3/4	6/8	7/8	5/4	12/8	2/4
steps per bar, $u = 1/8$	8	6	6	7	10	12	4
steps per bar, $u = 1/16$	16	12	12	14	20	24	8
cells, $u = 1/16$, 2 bars	160	120	120	140	200	240	80
cells, $u = 1/16$, 4 bars	320	240	240	280	—	—	160

DECISION D1 · AGREED How large is the grid, and what are empty cells?

Options. (a) The host fixes the shape and a capacity $N \leq 5S$ at creation, and unseated cells are void. (b) The grid grows by a bar when it fills. (c) The grid has exactly as many cells as there are players.

Agreed: (a). A groove is planned against a fixed loop, and (b) and (c) would change its length during play. Void cells are silence, and silence is most of any good beat: a full grid is a wall of sound. The default is 4/4 in sixteenth-note columns, 2 bars, and a capacity of 32, so one cell in five can sound. The bound is $S \leq 64$ steps (320 cells) in this version, so that a whole-grid read stays one explore deploy of modest cost. Phase 4 of the plan measures it.

3.3 Seating

DECISION D2 · AGREED Who chooses where a player sits?

Options. (a) Random: the environment assigns a cell. (b) Claim: the player chooses a free cell. (c) Row: the player chooses an instrument, and the environment assigns a free step in that row at random.

Agreed: (a) by default, with (b) and (c) as host options. Random seating keeps where a player sits out of their hands, as in F1R3Pix, so neighbours who have never met must still negotiate. Row seating is new and suits music: a bassist can choose to play bass without choosing their moment. Claim seating suits games among friends and tournaments with set formations.

Random seating, precisely. As in F1R3Pix, with cell indices in place of spiral indices. Let $h = \text{blake2b256}(\text{instance} \parallel \text{address})$, read as an unsigned integer. Starting at index $h \bmod 5S$, the environment takes the first free cell in increasing index order, wrapping at the end. Row seating does the same within the row's cells $\{5t + \rho\}$, starting at step $h \bmod S$. Either result can be checked by anyone.

DECISION D3 · AGREED What happens to a cell when its player leaves?

Agreed: as F1R3Pix D3. The cell stays owned and keeps playing its last note. The client draws it dimmed, and rejoining restores control.

3.4 Pitch

DECISION D4 · SETTLED A cell plays one pitch from its row's palette

Settled on 6 October 2026: as in F1R3Pix, where a cell takes a colour, a cell in F1R3Beat takes a pitch from a palette. The drum set is therefore a single row, whose palette is the pieces of the kit. That also reduces the grid's height from the prototype's eight drum rows plus four instruments to five rows.

What follows from it. A pitched row's palette is every pitch in its range. The host may restrict the pitched rows to a scale, written as F1R3Score writes one: a kind (major, minor, dorian, pentatonic, minor-pentatonic, blues or chromatic) and a tonic. A row's palette is then the scale's pitches within the row's range. The drum row is unaffected by the scale. Pitches are written in scientific notation with sharps (C#3), as F1R3Score names MIDI numbers. Kit pieces are written by name. One cell plays one note, so each row is monophonic, and harmony comes from the rows sounding together.

name	piece	MIDI	name	piece	MIDI	name	piece	MIDI
kick	bass drum	36	chh	closed hi-hat	42	ltom	low tom	45
rim	side stick	37	phh	pedal hi-hat	44	mtom	low-mid tom	47
snare	snare	38	ohh	open hi-hat	46	htom	high tom	50
clap	hand clap	39	crash	crash cymbal	49	ride	ride cymbal	51

Table 1: The kit. Names are F1R3Score pitch identifiers with explicit MIDI numbers on the General MIDI percussion channel.

3.5 How long a note lasts

DECISION D5 · SETTLED A note lasts one column, and the column is set when the game begins

Settled in review on 6 October 2026: note length is decided by column size. A column determines the subdivision of the beat, and it is chosen when the game is set up, together with the time signature and the number of bars. Every note lasts exactly one column.

What follows from it. No row holds its notes, and no palette contains a rest: a cell plays one pitch or nothing. A longer sound is a choice of subdivision, not a property of a row, so a host who wants longer notes chooses a coarser column. The map from a grid to its score is injective (R3). The column is part of a pattern's shape, so it joins the meter in deciding which patterns can breed (D13), and a seed must match it (D16). Notes longer than one column, written as ties across cells, are left for a later version.

3.6 Phases

DECISION D6 · AGREED What may players do in each phase?

Agreed: as F1R3Pix D5, with listen beside seat.

	seat	set	listen	message	pay	read
lobby	yes	no	yes	yes	yes	yes
active	yes	yes	yes	yes	yes	yes
closed	no	no	no	no	no	yes

3.7 Tempo

DECISION D7 · SETTLED Tempo is local to each listener

Settled on 6 October 2026: an instance has a default tempo, but changing the tempo is a local move. Player A can listen at a different tempo from player B. The tempo a player is listening at is a description of the player: *listening at 92 bpm*.

What follows from it. Tempo is not part of the pattern. It is not in the grid or in the score, and two patterns that differ only in the tempo they were heard at are the same pattern. This matches F1R3Score, in which a score is written in whole notes and - - bpm is a rendering parameter only. The host sets the default (100 bpm unless changed, within 40–240). Each player's listening tempo is published on the chain, so that it can be seen, by the `listen` move (§5). A published pattern carries its publisher's listening tempo as a suggestion, and every listener can override it. The installed tempo move is retired.

4 ARCHITECTURE

F1R3Beat adds one new kind of component to what F1R3Pix uses: a *breeder*, which runs the gallery's evolution. Everything else is one game environment on the shard, one game client framed by the Portal shell, and the Portal's existing payments domain and host protocol 2. The score bridge, the code that turns a grid into a score, is a library shared by the client, the breeder and anyone who audits them.

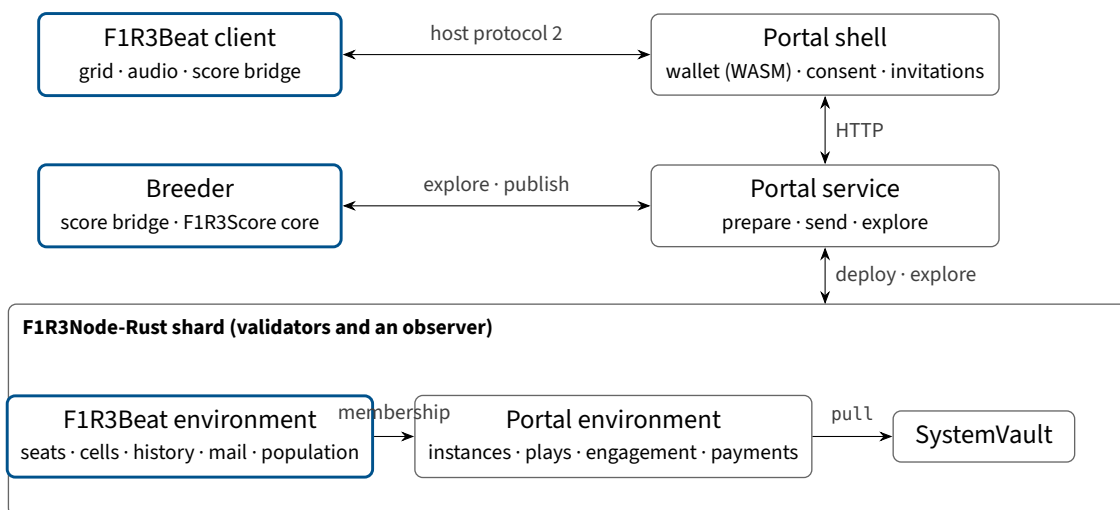


Figure 1: Components. Outlined in dark sky: what this design adds or replaces.

4.1 Authority

The design keeps the Portal's separation of authority, as F1R3Pix does.

- The game client never sees a key. It reaches identity, the shard and payment only through the host protocol.
- The F1R3Beat environment reads two things and nothing else: the caller's address, which it obtains from `rho:deploy:data` through `rho:vault:address`, and the block height, from `rho:block:data`. It

never touches a vault.

- Funds move only through the Portal environment's payments . send, signed only from the Portal shell after a prompt.
- The breeder holds one key, the *breeder key*, registered in the F1R3Beat environment by the F1R3FLY.io Cooperative. That key can admit patterns to the population and remove them, and nothing else. Everything it does is recomputable from public data (§10).

5 THE F1R3BEAT ENVIRONMENT

The environment replaces the installed `f1r3beat.rho` body inside the shared game `prelude.rho`, keeping the prelude's machinery (installation under the game's registry key, version-checked upgrade, who, member, `instanceOf`, `TreeHashMap`). Its moves follow F1R3Pix's moves one for one. `set` plays the part of paint, and `listen` is new. Installing it raises the environment version. The URI is unchanged and every call template's hash changes, so the manifest is re-registered (§12).

5.1 Configuration

The configuration is the config map the host passes to the Portal's instances . create. The environment reads it from the Portal's instance record.

```
{ "meter":    [4, 4],           // n/d, with n/(d*column) a whole number
  "bars":     2,               // S = n*bars/(d*column) steps, 1 <= S <= 64
  "capacity": 32,              // 1 ..= 5*S                               (D1)
  "seating":  "random",        // "random" | "claim" | "row"                               (D2)
  "scale":    Nil,             // Nil, or ["minor-pentatonic", "E"]                         (D4)
  "column":   [1, 16],         // 1/4, 1/8, 1/16, 1/32, 1/6, 1/12, 1/24                     (D5)
  "tempo":    100,             // default listening tempo, 40 ..= 240                       (D7)
  "seed":     Nil,             // Nil, or a seeding pattern                                 (D16)
  "messageLimit": 2048 }      // bytes of sealed envelope per message
```

R1 The environment MUST refuse every move on an instance whose configuration is missing or out of range, answering (false, "bad config"). It MUST NOT substitute defaults. Defaults are applied by the client that creates the instance, where the host can see them.

5.2 State

key	value	written by
("seat", i, a)	{"cell": c, "pk", "listen": bpm, "at", "h"}	seat, listen, by <i>a</i>
("cell", i, c)	{"owner": a Nil, "note": v, "at", "h", "n"}	seat, set
("hist", i, c)	list of [h, ts, v], oldest first	set
("out", i, a)	sealed envelopes, as F1R3Pix	say, by <i>a</i>
("pop", p)	{"meter", "steps", "born", "origin"}	breeder key

A cell's note v is Nil (plays nothing) or a pitch name from the row's palette. Seeded cells that no one occupies have owner Nil and keep their seeded note (D16).

R2 After seating, every key ("cell", i , c), ("hist", i , c), ("seat", i , a) and ("out", i , a) has exactly one writer. Players setting notes, changing tempo or messaging never contend for a key. There is deliberately no instance-wide counter or log.

5.3 Moves

Moves are play templates, signed within the allowance granted at launch. Each answers (true, value) or (false, reason).

seat(instance, pk, want). As F1R3Pix's seat. want is Nil under random seating, a cell index under claim seating, or a row index under row seating. The key pk is checked against the caller's address with rho:vault:address's "fromPublicKey". A new seat's listen is the instance's default tempo. Seating on a seeded cell takes ownership of it without changing its note. Answers the cell index.

set(instance, note). Sets what the caller's own cell plays. Like paint, the move names no cell.

- Refuses unless the caller is a participant, the status is active, and the caller is seated.
- Refuses unless note is Nil or a member of the palette of the caller's row (D4).
- Writes the cell with n incremented, appends [h , ts , note] to its history, and answers {"cell": c , "note": note, "h": h }.

listen(instance, bpm). Records the tempo the caller is listening at. It refuses unless the caller is seated, the status is not closed, and $40 \leq bpm \leq 240$ is an integer. It changes nothing anyone hears but the caller. The client debounces it, so it publishes a tempo only once the slider has rested for two seconds.

say(instance, to, envelope). As F1R3Pix's say.

The breeder key has two further moves, described in §10: admit and cull.

5.4 Reads

template	answers
f1r3beat.grid(instance)	the configuration, the status, and every non-void cell with owner, note, at, h and n
f1r3beat.seats(instance)	map from address to {"cell", "pk", "listen"}
f1r3beat.log(instance, from, to)	every set with $from \leq h < to$, as [h , ts , owner, c , v], in history order
f1r3beat.mail(instance, address, cursor)	as F1R3Pix
f1r3beat.outbox(instance, address, from)	as F1R3Pix
f1r3beat.population(cursor)	population members after cursor, with their records

Every read result carries the block hash it reflects. History is ordered by height, then timestamp, then owner address, then the order of sets within a cell, as in F1R3Pix.

5.5 A sketch of set

```
contract env(@set, @instance, @note, ret) = {
  new whoCh, rCh, sCh, hCh, okCh, ack1, ack2 in {
    who!(*whoCh) |
    for (@by, @ts <- whoCh) {
      ready!(instance, by, "active", *rCh) |      // participant, status, config
      for (@ok, @why <- rCh) {
        if (not ok) { ret!((false, why)) } else {
          TreeHashMap!("get", state, ("seat", instance, by), *sCh) |
          for (@seat <- sCh) {
            match seat {
              Nil => { ret!((false, "take a seat first")) }
              _ => { match seat.get("cell") { c => {
                allowed!(instance, c, note, *okCh) |    // palette of row c % 5
                for (@fits <- okCh) {
                  if (not fits) { ret!((false, "not in this row's palette")) } else {
                    height!(*hCh) |
                    for (@h <- hCh) {
                      update!("cell", instance, c), note, ts, h, *ack1) |
                      append!("hist", instance, c), [h, ts, note], *ack2) |
                      for (_ <- ack1 & _ <- ack2) {
                        ret!((true, {"cell": c, "note": note, "h": h}))
                      }
                    }
                  }
                }
              } } }
            }
          }
        }
      }
    }
  }
}
```

allowed computes a row's palette from the configuration once per instance and keeps it under ("palette", i, row), written at the instance's first seat. The palette of a pitched row is at most 49 pitches, so the check is a set lookup.

6 MESSAGES AND PAYMENTS

DECISION D8 · AGREED How do players talk and pay?

Agreed: exactly as F1R3Pix (its D6, D7 and D10). Messages are on the chain and sealed to their recipients and the sender, with the domain strings "f1r3beat/msg/v1" in place of F1R3Pix's. Payments go through the Portal environment's payments.send, signed only from the shell after a prompt, with *each* as the default and *split* on request. The manifest declares "capabilities": ["pay", "open"]. Nothing in either mechanism is specific to F1R3Pix, and reusing it means the second game to use it proves that.

The same requirements apply: the host binds the game id and instance into open (F1R3Pix R4), refuses a pay naming a non-participant before prompting (F1R3Pix R7), and never signs payments . send within an allowance (F1R3Pix R6).

7 FROM GRID TO SCORE

DECISION D9 · SETTLED Patterns are written, and reproduce, in F1R3Score's notation

Settled on 6 October 2026: the recombination operator for F1R3Beat uses the notation system of F1R3Score. This section gives the encoding. §10 defines reproduction on it.

7.1 The shape of the score

A pattern is the parallel composition of five lines, one per row. Each line is the sequence of the notes and rests that its row plays, in F1R3Score's line form: a right-nested chain of written notes, each a key on a pitch channel and a touch on a duration co-channel that meet and play. The components in composition are therefore the instruments, and the composition is the beat. Each line's durations add up to the length of the pattern, so the five lines start together and end together, and the pattern loops.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
drums	kick		chh		snare		chh		kick		kick		snare		ohh	
bass	E2			E2			G2		A2			A2				
guitar					E3								D3			
keys																
sax					D4				E4							

Figure 2: A pattern in 4/4, one bar, in sixteenth-note columns. Each sounding cell is a note one column long. The keys row plays nothing.

```
// F1R3Beat pattern, canonical form v2 · meter 4/4 · bars 1 · column 1/16 · columns 16
score F1R3Beat
import std
pitches { r, kick = 36, snare = 38, chh = 42, ohh = 46, E2, G2, A2, D3, E3, D4, E4 }
durations { c1 = 1/16, c2 = 1/8, c3 = 3/16, c4 = 1/4, c7 = 7/16, c16 = 1 }
timbres { drums = gm(0) on 10, bass = gm(33) on 2, guitar = gm(29) on 3,
          keys = gm(0) on 4, sax = gm(66) on 5 }
play line(base "drums", drums) [ kick c1, r c1, chh c1, r c1, snare c1, r c1, chh c1, r c1,
                                kick c1, r c1, kick c1, r c1, snare c1, r c1, ohh c1, r c1 ]
  | line(base "bass", bass) [ E2 c1, r c2, E2 c1, r c2, G2 c1, r c1, A2 c1, r c2,
                             A2 c1, r c4 ]
  | line(base "guitar", guitar) [ r c4, E3 c1, r c7, D3 c1, r c3 ]
  | line(base "keys", keys) [ r c16 ]
  | line(base "sax", sax) [ r c4, D4 c1, r c3, E4 c1, r c7 ]
```

This is the pattern of Figure 2. f1r3score check accepts it (digest 1fc69fbb...), and f1r3score play plays every note at its column: kick and bass at 0, the closed hat at 1/8, the second bass note at 3/16, snare, guitar and saxophone at 1/4, and so on.

7.2 Canonical form

Breeding, deduplication and verification all compare patterns, so the score of a grid must be unique. F1R3Score's structural digest depends on the declarations as well as on the play term, including the order of the alphabets. The *canonical form* fixes every choice.

1. **Lines.** For each row, in the order drums, bass, guitar, keys, sax, read the cells $v_0, \dots, v_{S-1}$. Each pitch v_t is the note (v_t, u) . Each maximal run of silent cells is one rest r , as long as the run. A row with no notes is the single rest $[r \ cS]$.
2. **Durations.** A length of k columns is named $ck = ku$, written in lowest terms. Exactly the lengths used are declared, in increasing order.
3. **Pitches.** r first; then the kit pieces used, in increasing MIDI order (Table 1); then the pitched notes used, in increasing MIDI order, in scientific notation with sharps.
4. **Timbres.** All five declarations, always, as in the table of §3.
5. **Text.** The score is named F1R3Beat and imports std. The first line is the comment shown above. Layout is the one shown: two spaces of indentation, `|` at the start of each line after the first, and a single space after each comma.

R3 The score bridge MUST produce the canonical form, and a pattern's identity is the F1R3Score digest of its canonical score. Because every note is one column long, two grids of one shape have the same digest exactly when they play the same notes.

7.3 What the score is used for

- **The gallery.** A pattern play's body is its canonical score (§9), so a published pattern can be played by `f1r3score` with no F1R3Beat code at all, and rendered to MIDI with `f1r3score play --midi`.
- **Identity.** The digest names the pattern. It deduplicates the population and links a child to its parents.
- **The genome.** Reproduction is defined on lines and on the composition of lines (§10).
- **Playback.** The client plays the notes the score denotes. Conformance tests check that the client's note list is the one `f1r3score play --format json reports`, rests excluded (§12).

7.4 Why lines and not keyboards

F1R3Score's instruments separate the player from the instrument: a key, holding a pitch, belongs to the instrument, and a touch, holding a duration, belongs to the player. That reading fits a game of players at instruments, and it was the first encoding tried. As Finding 7 records, it does not keep a grid's time, because touches waiting at one key contend whatever their stamps. A line is sequential composition. Each note's continuation is stamped at the end of the note before it, so time is exact.

What the line form loses is a record, inside the score, of which player played which note. The gallery keeps that separately, as the play's credits (§9). When F1R3Score gains a timed touch, the keyboard form can be revisited (§15.1).

8 THE CLIENT

The client follows F1R3Pix's: a web page from the game's own origin, framed by the Portal, with a framework-free core holding every operation and a React layer that only renders. The core adds two modules to F1R3Pix's: the TypeScript score bridge (grid to canonical score, and score to notes), and the audio engine.

8.1 Layout

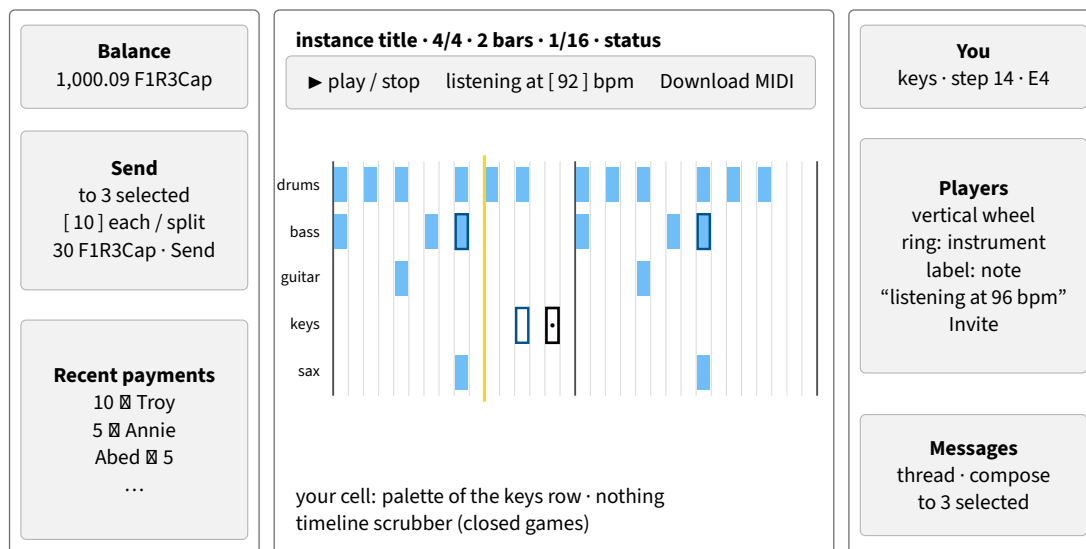


Figure 3: The three columns. In the grid, the black-outlined cell with a dot is yours, dark sky outlines mark the selected players' cells, and the yellow line is the playhead, which moves at your own tempo.

Left column: tokens. As F1R3Pix: balance, the send control (*each* or *split*, with the total), and the instance's recent payments.

Centre column: the grid. Above it, the transport: play and stop, the player's listening tempo, and *Download MIDI*. The grid shows rows labelled by instrument, bar lines heavy, beat lines medium and columns light. Sounding cells carry their note's name. Void cells are hatched, seeded cells that no one has taken have a dotted outline, and the cells of players who have left are dimmed. The player's own cell is marked independently of colour, with a black outline and a dot. Clicking it opens the note control. For the drum row, this is the twelve pieces as buttons. For a pitched row, it is a keyboard strip spanning the row's range, with the scale's pitches enabled and the rest disabled, plus *nothing*. Hovering over a choice plays it locally, which costs nothing and deploys nothing. Clicking another player's cell toggles their selection, so the grid is also a selection surface.

Right column: players. As F1R3Pix. Each avatar's ring shows the player's instrument, its label shows their current note, and its caption shows the tempo they are listening at (D7). Selection drives board highlighting, message recipients and payment recipients together.

DECISION D10 · AGREED In what order does the wheel list players?

Agreed: by distance on the grid, with time wrapping, then by name or by recent contact on request. The distance between cells (t, ρ) and (t', ρ') is $\min(|t - t'|, S - |t - t'|) + |\rho - \rho'|$, measured on the loop as a cylinder. The players nearest a player sound at the same moment, which is where hits and chords are made, or just before and after in the same row, which is where lines are made. Those are the players they most need to talk to.

8.2 Sound

- **Rendering.** After each grid read, the core rebuilds the canonical score and its note list (timbre, pitch, onset, length, rests excluded), and the audio engine schedules the loop on the Web Audio clock at the listener's tempo. Tempo counts quarter notes, as F1R3Score's renderer does, so a column of u whole notes lasts $240u/bpm$ seconds. Changes take effect at the next loop boundary, so the groove never stutters mid-bar.
- **Timbres.** Everyone must hear the same instruments, so the client bundles one General MIDI sound set: the four programs and the kit, and nothing more. It MUST be permissively licensed. FluidR3_GM (MIT licence) is the candidate, cut to those samples. A device's own synthesizer is not used, because it would make the same pattern sound different to different players.
- **Tempo.** The tempo control is local and immediate. It publishes through `listen` only after the control has rested (D7).
- **MIDI.** `Download MIDI` writes the loop as a Standard MIDI File at the listener's tempo, one track per timbre, matching what `f1r3score play --midi` writes for the same score.

8.3 Behaviour

- **Freshness, pending sets and allowance.** As F1R3Pix, with `set` in place of `paint`. A note change sounds locally at once with a pending mark, and settles when a block includes it. While a set is in flight, further choices replace a single queued set. The allowance covers `seat`, `set`, `listen` and `say`.
- **Accessibility.** No information is carried by colour alone: note names are printed in their cells, the player's own cell has an outline and a dot, and void, seeded and departed cells have hatching, dotting or dimming. Keyboard navigation follows cell indices, which is time order. The grid can be read aloud row by row, as note names and step numbers.
- **Small screens.** Below 900 pixels, the columns become tabs, with the grid as the default tab. A grid wider than the screen scrolls horizontally, and the playhead keeps the current bar in view.

9 GALLERIES

DECISION D11 · AGREED What does a F1R3Beat play hold?

Options. (a) Only patterns, as installed. (b) Only the history of a game. (c) Both, as two gallery kinds, linked.

Agreed: (c). F1R3Skein distinguishes a *performance* from the *tunes* a performer snips from it, and F1R3Beat has the same structure. A session is the whole history of a game over a span of blocks, like a F1R3Pix play. A pattern is the grid at one block, written as its canonical score. Patterns are what breed. Any participant may cut a pattern from a session at any block, and the two are linked with linkPlays. When the host closes an instance, the client offers to publish the session and its final pattern together, with one prompt.

9.1 Patterns

part	content
header	title; digest (R3); meter; column; steps; tempo, the publisher's listening tempo as a suggestion; origin, one of game, bred or cross; parents, the play ids of both parents for bred and crossed patterns, or of the seed for a seeded game; operator (X or V, §10); seed, the block hash and epoch from which the generator was drawn; for a game pattern, instance and blockHash
body	CBOR {score: <canonical score text>, credits: [[cell, address], ...]}. Credits name the player seated at each sounding cell. Bred patterns have none, and their lineage credits their parents.

A pattern is verifiable. A game pattern's score can be rebuilt from `f1r3beat.grid` at `blockHash`. A bred or crossed pattern can be rebuilt from its parents and its seed (§10.7). In both cases its digest must match. Every published score plays in `f1r3score` with no F1R3Beat code at all. The preview renderer draws the grid from the score, and the full view plays it.

9.2 Sessions

A session's body is a byte array, the F1R3Pix encoding adapted to cells and notes:

```
"F1BT"  u8 version = 1  u8 n  u8 d  u8 k  u8 bars  u32 from  u32 to  (column = 1/k)
u8 P  P × (u8 row, u8 code)      notes used in the span, in first-use order;
                                   code is the MIDI number
varint S  S × (varint Δh, u16 cell, u8 note)  sets in history order;
                                   note indexes the table, 255 = nothing
varint 0  0 × (u16 cell, 20 bytes address)  owners of the cells that appear
```

The full view plays a session as it was played: the loop runs, and the grid changes block by block beneath the playhead.

9.3 Engagement

Views, plays and likes go through the Portal's engage. For sessions they rank the gallery. For patterns they also set selection weights (D14).

10 EVOLUTION

10.1 The population

The *population* is a set of patterns kept in the environment under ("pop", p). Membership is separate from publication: a pattern that leaves the population stays in the gallery. Patterns join in three ways. A pattern published from a game joins at the next epoch. A bred pattern joins when it is admitted. A crossed pattern joins once someone other than the player who crossed it likes it (D15).

10.2 Lines as streams

Write a pattern of S steps as its five lines $P = (P_\rho)_{\rho \in R}$, where $R = (\text{drums}, \text{bass}, \text{guitar}, \text{keys}, \text{sax})$, and a line as $L = [(p_1, d_1), \dots, (p_k, d_k)]$ with $\sum_j d_j = Su$. Following F1R3Skein, a line unzips into two streams. In F1R3Beat the streams are cut differently, because the grid fixes time:

- the *rhythm* $\text{rh}(L) = [(d_j, [p_j \neq r])]_j$, the durations together with which of them sound. It has $n(L) = \#\{j : p_j \neq r\}$ slots;
- the *melody* $\text{mel}(L)$, the subsequence of the pitches $p_j \neq r$. For the drum row it is the sequence of kit pieces.

Refilling. Given a rhythm with n slots, a melody of length k and a generator g , $\text{refill}(\text{rh}, \sigma, g)$ is a line. If $k \geq n$, delete $k - n$ entries of σ , chosen uniformly by g , keeping the order of the rest. If $k < n$, choose $n - k$ slots uniformly by g and make them rests. Then assign the melody to the remaining slots in order, and normalise as in §7.2, merging adjacent rests.

10.3 The operators

DECISION D12 · AGREED What are the children of two patterns?

Agreed: four children, two from each of two operators, as F1R3Skein has four. Agreed in review as workable for now.

Rhythm and melody, row by row. The cross of A by B keeps A 's rhythm and takes B 's melody, independently in each row:

$$X(A, B)_\rho = \text{refill}(\text{rh}(A_\rho), \text{mel}(B_\rho), g).$$

This is F1R3Skein's first two children (Mother's pitches with Father's durations, and the converse) applied to every line of the composition. A child has the length of its rhythm parent. Every pitch stays in its row's range, because a melody never leaves its row.

Voices exchanged at the composition. For a vector $c \in \{0, 1\}^5$ that is neither all zeros nor all ones,

$$V(A, B; c)_\rho = \begin{cases} A_\rho & c_\rho = 0 \\ B_\rho & c_\rho = 1 \end{cases}$$

takes whole lines from each parent. This operator cuts the score exactly where it composes, at the parallel bars, where harmony comes from independence. It keeps each line intact, so a bass figure and the kick it locks with can be inherited together if they come from the same parent.

The brood of A and B is $\{X(A, B), X(B, A), V(A, B; c), V(B, A; c)\}$, with one c drawn for both voice children, so the two are complements.

Why not F1R3Skein's other two children. F1R3Skein's third and fourth children rewrite one parent's pitch stream as durations (in base 5) or its durations as pitches (in base 22). On a grid, a duration must be a whole number of steps and a line's durations must fill the pattern exactly, so rewritten digits would break the loop. The second axis of a beat is not a change of alphabet but the composition of instruments, and *V* recombines along it.

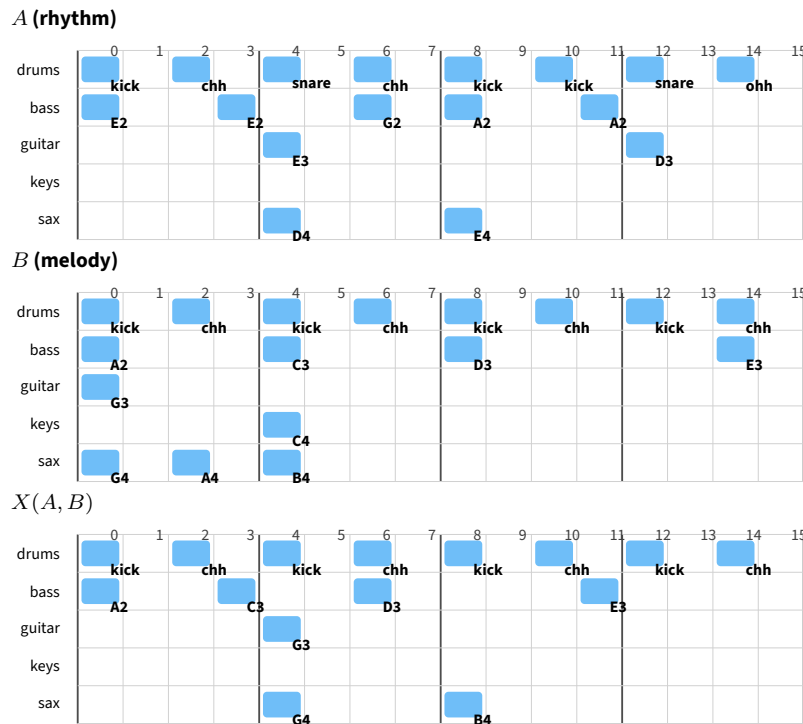


Figure 4: A cross. *A* is the pattern of Figure 2. The child keeps *A*'s rhythm in every row and takes *B*'s melody: the drums keep *A*'s eight hits with *B*'s alternation of kick and hat, The bass has five slots and *B*'s bass melody four pitches, so one slot, drawn by *g*, falls silent, and so does one of the guitar's two slots. The saxophone has two slots and *B*'s saxophone melody has three pitches, so one pitch, drawn by *g*, is dropped. The keys have no slots, so *B*'s keys melody is lost.

The child of Figure 4, in canonical form, is accepted by `f1r3score check (digest fa30e9ae...)`:

```
play line(base "drums", drums) [ kick c1, r c1, chh c1, r c1, kick c1, r c1, chh c1, r c1,
                                kick c1, r c1, chh c1, r c1, kick c1, r c1, chh c1, r c1 ]
  | line(base "bass", bass)    [ A2 c1, r c2, C3 c1, r c2, D3 c1, r c4, E3 c1, r c4 ]
  | line(base "guitar", guitar) [ r c4, G3 c1, r c11 ]
  | line(base "keys", keys)    [ r c16 ]
  | line(base "sax", sax)      [ r c4, G4 c1, r c3, B4 c1, r c7 ]
```

10.4 Shapes and mutation

DECISION D13 · AGREED Which patterns can breed, and where does variation come from?

Agreed.

Shapes are species. Two patterns breed only if they have the same time signature and the same column (D5). Lines in different meters or subdivisions do not share a grid, and splicing them would yield polymeter. That is interesting, but it is a different game (§15.1).

Lengths are equalised by silent bars. X needs no equalising, because the child takes its rhythm parent's length. V needs both parents to be the same length. The shorter is padded with silent bars, inserted at bar boundaries chosen uniformly by g , until the lengths match. This is F1R3Skein's padding of the shorter snippet with rests at random places, moved to the bar, which is the unit at which a beat can grow without breaking its pulse.

Mutation. Variation comes from refilling (slots silenced, pitches dropped) and from padding, as it does in F1R3Skein, and from nothing else by default. A point mutation, which replaces a sounding pitch with one drawn uniformly from its row's palette at rate μ per note, is specified with $\mu = 0$. It can be turned on once Phase 6 shows whether the population loses diversity without it.

10.5 Selection and culling

DECISION D14 · AGREED Which patterns breed, and which leave?

Agreed. The weight of a pattern is $w = 1 + \text{plays} + 3 \cdot \text{likes}$, counted from the Portal's engagement by distinct addresses, as the F1R3Skein prototype counted it. At each epoch:

1. Draw A from the population with probability proportional to w .
2. Draw B , proportionally to w , from the members of A 's species other than A . If there are none, draw A again from the species with at least two members, and if there are none, the epoch breeds nothing.
3. Compute the brood. Drop any child whose digest is already in the population, or equals another child's.
4. Admit the brood. Then draw k uniformly from $\{1, 2, 3\}$, and remove the k members with the least weight among those at least three epochs old. Ties go to the older member, then to the smaller digest. The population is never culled below 16 members.

The grace period is new. Without it, a child, whose engagement starts at nothing, is the likeliest member to be culled at the very epoch that admits it, so selection could never see it.

10.6 Who breeds

DECISION D15 · AGREED Who runs reproduction?

Options. (a) A breeder service runs epochs. (b) Players cross patterns they choose. (c) Both.

Agreed: (c). Epochs run every E blocks, with E set in Phase 4 so that an epoch is about a day. The breeder computes the epoch from public data and signs two moves with the breeder key: `admit(epoch, children)` and `cull(epoch, members)`. The environment checks only the key and that epochs increase. Correctness is checked by anyone who recomputes (§10.7). The breeder key is registered by the F1R3FLY.io Cooperative, which governs the registry.

Crosses let a player choose two patterns of the same species in the gallery and breed them. The client computes the brood and publishes the four children, with origin cross, in one prompt. A crossed child enters the population only after someone other than the crosser likes it. That keeps the population from filling with one player's crosses, and it means a human other than the crosser has heard the child before it can breed.

10.7 Randomness and verification

R4 The generator g for an epoch MUST be xoshiro256** seeded through SplitMix64, as F1R3Score's score-chance crate implements it. The seed MUST be the first eight bytes, read little-endian, of

$$\text{blake2b256}(\text{"f1r3beat/breed/v1"} \parallel H \parallel \text{epoch}),$$

where H is the hash of the epoch's block, and engagement MUST be read as of that block. For a brood, the seed is extended with the two parents' digests. For a cross, H is a block named in the children's headers, which MUST be at most 50 blocks older than the block that includes the publication.

R5 Draws MUST be taken in this order: selection; padding positions, in increasing bar order; c , as a uniform integer in $[1, 30]$ read in binary with the drum row as the most significant bit; then the refills of $X(A, B)$ in row order, then those of $X(B, A)$. A uniform integer below m is drawn by rejection: take a 64-bit output x , retry while $x \geq m \lfloor 2^{64}/m \rfloor$, and answer $x \bmod m$. A uniform choice of j positions out of m is the first j entries of a Fisher–Yates shuffle of $0, \dots, m - 1$, sorted.

Given those rules, an epoch is a pure function of the chain. `f1r3beat-breed verify --epoch e` recomputes it from the population, the engagement and the block hash, and compares the digests admitted and culled with those the breeder key wrote. A breeder that cheats is caught by anyone who runs it.

10.8 Patterns back into play

DECISION D16 · AGREED Can a game start from a pattern?

Agreed: yes. A host may seed a new instance from any pattern of the same shape: meter, column and length. The client converts the score into cell notes, which is exact because every note is one column long. The configuration's seed carries the play id, the digest and the cell notes. The environment writes them as cells with no owner. A player seated on a seeded cell inherits its note and can change it. Patterns published from the game list the seed as their parent.

This closes the loop. Players make patterns, the gallery's engagement selects among them, reproduction recombines them, and a recombined pattern can become the starting point for people to play again. The evolution never runs without humans in it.

11 SECURITY AND ABUSE

Everything in F1R3Pix's table applies, with set for paint. The rows below are the threats F1R3Beat adds.

threat	answer
Setting another player's cell	Impossible: set names no cell and writes only the caller's seat.
Injection through notes	The environment accepts only palette members or <code>Nil</code> . The score bridge writes score text only from fixed tables of row, pitch, kit and duration names, never from strings it was given.
A dishonest breeder	The breeder key can only admit and cull. Every epoch is recomputable (R4, R5), and <code>verify</code> exposes a forgery. The Cooperative can rotate the key.
Engagement farming	Weights count distinct addresses, and each address engages once per kind. Sybil addresses still cost phlo and a profile. Tournaments and stake-weighted engagement are the stronger answers, and are later work.
Grinding a cross's seed	The block named for a cross must be at most 50 blocks old, so a crosser chooses among at most 50 broods. A crossed child cannot breed until someone else likes it.
Flooding the population	Game patterns join once per publication. Crosses need another player's like. Culling keeps the size bounded.
Metadata	Listening tempo is public by design (D7). Message metadata is public as in F1R3Pix.

12 MANIFEST AND CONFORMANCE

```
{ "id": "f1r3beat", "name": "F1R3Beat",
  "tagline": "One step each. Make a groove together.",
  "entry": "<entry-base>/f1r3beat/", "platforms": ["web"],
  "env": "rho:id:...", // unchanged URI; environment version raised
  "templates": [ "f1r3beat.seat", "f1r3beat.set", "f1r3beat.listen", "f1r3beat.say", //
    play
                  "f1r3beat.grid", "f1r3beat.seats", "f1r3beat.log", "f1r3beat.mail",
                  "f1r3beat.outbox", "f1r3beat.population" ], //
    explore
  "capabilities": ["pay", "open"],
  "galleries": [ { "kind": "pattern", "renderer": "<entry-base>/f1r3beat/preview/pattern.
    html" },
                  { "kind": "session", "renderer": "<entry-base>/f1r3beat/preview/session.
    html" } ],
  "contactsDialogue": true, "readerTier": false, "minProtocol": 2 }
```

The templates `f1r3beat.toggle`, `f1r3beat.tempo` and `f1r3beat.state` are retired. `admit` and `cull` are not game templates. They are signed only by the breeder key, from the breeder, and are absent from the manifest.

Conformance tests extend the Portal's and F1R3Pix's:

- the environment and every template parse with the node's own parser (`rho:lint`), and every play template binds no deployer authority;
- cell indices, seating (random, claim and row), palettes and session encodings round-trip against shared vectors in Rust and TypeScript;
- for every grid vector, the Rust and TypeScript bridges produce the same canonical text. `f1r3score` check accepts it with the recorded digest, and the notes of `f1r3score play --format json`, rests excluded, are exactly the onsets the grid denotes. This runs in CI against F1R3Score pinned at a stated revision;
- breeding vectors: for given parents, block hash and epoch, the brood's digests, in Rust and TypeScript. `verify` accepts a correct epoch and rejects one with a single child altered;
- end to end against the mock node: seat, set, listen, say, open, pay, publish a pattern and a session, rebuild both from the chain, run an epoch, and verify it.

13 F1R3GAZE

F1R3Beat needs the two capabilities F1R3Pix adds to the Portal's GAZE-MAPPING.md: `pay`, attenuated to the instance's participants, and `open`, bound to the game and instance. It needs one more, which no game has needed so far: *sound*. A `f1r3lang` page in F1R3Gaze has no audio output. The natural capability takes

note records (timbre, pitch, onset, length) and plays them against the browser's clock at a given tempo. Those are the records F1R3Score's renderer already produces, and its live-MIDI path already schedules them in wall time. A sound capability shaped that way would let a f1r3lang page play any F1R3Score performance, not only F1R3Beat's. GAZE-MAPPING.md should gain the row when Phase 3 lands.

14 DEVELOPMENT PLAN

phase	work	done when
1 · Environment	New f1r3beat.rho body; seat, set, listen, say; reads; palettes; manifest; seating and palette vectors.	Parses; template rules pass; mock-node tests of every move and read.
2 · Score bridge	Rust crate (grid to canonical score, score to notes, MIDI) and its TypeScript twin; grid vectors; the CI job against f1r3score.	Both bridges agree on every vector, and f1r3score agrees with both.
3 · Client	Core and views; grid, note control, transport; audio engine and the cut-down sound set; pending sets; accessibility; tabs.	Plays a full game against the mock node with five simulated players, audibly.
4 · Live shard	Install; measure phlo for each move and read; set the allowance, the 64-step bound and the epoch length E .	One live game of at least eight players, start to close.
5 · Galleries	Pattern and session encodings; publishing at close; cutting patterns from sessions; renderers; engagement.	A closed game publishes both kinds, and both rebuild from the chain.
6 · Evolution	Breeder crate on the score bridge and F1R3Score's score-chance; population moves; crosses in the client; verify; seeding instances from patterns.	Ten epochs on a test shard, each verified independently; a seeded game played from a bred pattern.
Later	Uploaded samples and timbres; polimeter; a timed-touch keyboard form; tournaments; the F1R3Gaze version.	

Phases 1 and 2 can proceed together, and Phase 3 can begin against stubs. One dependency outside this repository constrains the plan. F1R3Score's DESIGN.md records that its wasm32 build has not been verified. Until it is, the client runs the TypeScript bridge, Rust runs only in the breeder and in verify, and the CI job of Phase 2 keeps the two honest against f1r3score itself.

15 DECISIONS

	question	decision (all agreed in review, 6 October 2026)
D1	Grid size, empty cells	Shape and capacity fixed at creation (default 4/4 in sixteenths, 2 bars, 32 seats; at most 64 steps); unseated cells are void and silent.
D2	Seating	Random and verifiable by default; claim and row seating as host options.
D3	A player who leaves	Keeps the cell, still sounding, dimmed.
D4	Settled. Pitch	One pitch per cell from the row's palette; the drum set is one row whose palette is the kit; optional host scale.
D5	Settled. Note length	Every note lasts one column; the column, the subdivision of the beat, is chosen at setup.
D6	Phases	Lobby: seat, listen, talk, pay. Active: also set. Closed: read only.
D7	Settled. Tempo	A default per instance; each player listens at their own tempo, shown as part of their description; tempo is not part of the pattern.
D8	Messages and payments	As F1R3Pix, with F1R3Beat's domain strings.
D9	Settled. Notation	Patterns are F1R3Score scores, a parallel composition of five lines, in a canonical form whose digest is the pattern's identity.
D10	Wheel order	Nearest on the looped grid first.
D11	Galleries	session (history) and pattern (a moment as a score), linked.
D12	Children	Four: rhythm-by-melody crosses both ways, and two complementary voice exchanges; workable for now.
D13	Shapes and mutation	Same meter and column only; silent bars pad length; variation from refilling and padding; point mutation off.
D14	Selection and culling	Weight $1 + \text{plays} + 3 \cdot \text{likes}$; cull 1–3 of the least weighted after a three-epoch grace; floor of 16.
D15	Who breeds	Epochs by a recomputable breeder, and player crosses that enter on another player's like.
D16	Seeding	A game may start from a pattern; seeded cells are inherited by whoever sits there.

15.1 Out of scope for version 1

Uploaded samples and timbres, which the March design anticipates, together with instances that choose their rows from timbres stored on the shard. Also out of scope: notes longer than one column (ties across cells); a second pass on the reproduction operators (D12); chords and velocity within a cell; swing; polymeter and breeding across meters; tempo as a graded dimension of the score; a keyboard form of the score, once F1R3Score has a touch that waits for its stamp; tournaments; hidden recipient lists and reports, as in F1R3Pix; and the F1R3Gaze version.